

TRABAJO PRÁCTICO **COMPILADOR**

CONSIDERACIONES GENERALES

Es necesario cumplir con las siguientes consideraciones para evaluar el TP.

1. Cada grupo deberá desarrollar el compilador teniendo en cuenta:
 - Todos los temas comunes (Ver ANEXO TEMAS)
 - El tema especial asignado al grupo.
2. Se fijarán puntos de control con fechas y consignas determinadas

PRIMERA ENTREGA

OBJETIVO: Realizar un **analizador lexicográfico** utilizando la herramienta JFLEX. La aplicación realizada debe mostrar una interfaz gráfica que pueda utilizarse como IDE del compilador, en la cual se debe poder:

1. Ingresar código de nuestro programa manualmente en un cuadro de texto adecuado a tal propósito, por ejemplo dentro del lenguaje Java JTextArea.
2. Cargar un archivo con código y poder editarlo dentro del cuadro de texto.
3. Compilar el programa ingresado (análisis léxico) y mostrar un texto aclaratorio, identificando los tokens reconocidos por el parser u errores encontrados en el análisis. Las impresiones deben ser claras. Los elementos que no generan tokens no deben generar salida.

El material a entregar será:

- El archivo jflex que se llamará **Lexico.flex**
- Un archivo de pruebas generales que se llamará **prueba.txt** incluyendo la prueba del tema asignado al grupo
- Un archivo con la tabla de símbolos **ts.txt**
- Código fuente del proyecto
- Archivo JAR ejecutable

Todo el material deberá ser subido a algún repositorio GIT (Github, Gitlab, etc.) y el enlace al repositorio, con los comentarios necesarios de la entrega incluidos en la actividad creada dentro de la Plataforma Virtual UNLu.

Fecha de entrega: 05/10/22

SEGUNDA ENTREGA

OBJETIVO: Realizar un **analizador sintáctico** utilizando la herramienta JAVA CUP. La aplicación realizada debe mostrar una interfaz gráfica que pueda utilizarse como IDE del compilador, en este caso deberá permitir:

1. Cumplir mismos puntos que en la primer entrega
2. Compilar el programa ingresado (análisis sintáctico) y mostrar por pantalla un texto aclaratorio identificando las reglas sintácticas que va analizando el parser. Las impresiones deben ser claras. Las reglas que no realizan ninguna acción no deben generar salida.

El material a entregar será:

- El archivo jflex que se llamará **Lexico.flex**
- El archivo jcup que se llamará **Sintactico.cup**
- Un archivo de pruebas generales que se llamará **prueba.txt** incluyendo la prueba del tema asignado al grupo
- Un archivo con la tabla de símbolos **ts.txt**
- Código fuente del proyecto
- Archivo JAR ejecutable

Todo el material deberá ser subido a algún repositorio GIT (Github, Gitlab, etc.) y el enlace al repositorio, con los comentarios necesarios de la entrega incluidos en la actividad creada dentro de la Plataforma Virtual UNLu.

Fecha de entrega: 09/11/22

ANEXO TEMAS

TEMAS COMUNES

WHILE

Implementación de *While* utilizando el formato que el grupo desee

DECISIONES

Implementación de *IF* utilizando el formato que el grupo desee

ASIGNACIONES

Asignaciones simples $a := b$

TIPO DE DATOS

Constantes numéricas

- enteras (16 bits)
- reales (32 bits)

El separador decimal será el punto “.”

Ejemplo:

```
a ::= 99999.99
a ::= 99.
a ::= .9999
```

Constantes string

Constantes de 30 caracteres alfanuméricos como máximo, limitada por comillas (" ") ,de la forma "XXXX"

Ejemplo:

```
WRITE "@sdADaSJfla%dfg"
var ::= "HOLA MUNDO"
```

**Las constantes deben ser reconocidas y validadas en el *analizador léxico*, de acuerdo a su tipo.
Las constantes guardan su valor en tabla de símbolos.**

VARIABLES

Variables numéricas

Estas variables reciben valores numéricos tales como constantes numéricas, variables numéricas u operaciones que arrojen un valor numérico, del lado derecho de una asignación.

Variables string

Estas variables pueden recibir una constante string

Las variables no guardan su valor en tabla de símbolos.

Las asignaciones deben ser permitidas, solo en los casos en los que los tipos son compatibles, caso contrario deberá desplegarse un error.

COMENTARIOS

Deberán estar delimitados por `/$` y `$/` y podrán estar anidados en un solo nivel.

Ejemplo1:

```
/$  
IF (a <= 30)  
    b = "correcto" /$ asignación string $/  
ENDIF  
$/
```

Ejemplo2:

```
/$ Así son los comentarios $/
```

Los comentarios se ignoran de manera que no generan un componente léxico o token.

SALIDA

Las salidas se implementarán como se muestra en el siguiente ejemplo:

Ejemplo:

```
put "ewr"      /* donde "ewr" debe ser una cte string */  
put 99.999 /* donde 99.999 debe ser cualquier cte numérica */  
put var        /* donde var debe ser cualquier variable numérica */
```

CONDICIONES

Las condiciones para un constructor de ciclos o de selección, deberán ser comparaciones binarias que pueden estar ligadas por un único conector lógico (AND y OR)

```
(expresión) < (expresión)  
(expresión >= expresión) AND (expresión < expresión)  
(expresión >= expresión) OR (expresión < expresión)
```

DECLARACIONES

Todas las variables deberán ser declaradas dentro de un bloque especial para ese fin, delimitado por las palabras reservadas `DEVAR` y `ENDDECVAR` siguiendo el siguiente formato:

```
DEVAR  
    Línea_de_Declaración_de_Tipos  
ENDDECVAR
```

Cada *Línea_de_Declaración_de_Tipos* tendrá la forma: *Variable* : *Tipo*

Ejemplos de formato: `DEVAR`

```
a1 : FLOAT  
b2 : INT  
p  : INT
```

ENDDECVAR

PROGRAMA

Todas las sentencias del programa deberán ser declaradas dentro de un bloque especial para ese fin, delimitado por las palabras reservadas PROGRAM y END, siguiendo el siguiente formato:

```
PROGRAM
    Lista_de_Sentencias
END
```

La zona de declaración de variables deberá ser previa a la sección del programa.

TABLA DE SIMBOLOS

La tabla de símbolos tiene la capacidad de guardar las variables y constantes con sus atributos. Los atributos aportan información necesaria para operar con constantes y variables.

Ejemplo 1er ENTREGA (sin agregar tipos de datos)

NOMBRE	TOKEN	TIPO	VALOR	LONG
a1	ID		_	_
b1	ID		_	_
_hola	CTE_STR	_	hola	4
_mundo	CTE_STR	_	mundo	5
_30.5	CTE_F	_	30.5	_
_55	CTE_E	_	55	_

Tabla de símbolos

Ejemplo 2da ENTREGA (agregando tipos de datos)

NOMBRE	TOKEN	TIPO	VALOR	LONG
a1	ID	Float	_	_
b1	ID	Int	_	_
_hola	CTE_STR	_	hola	4
_mundo	CTE_STR	_	mundo	5
_30.5	CTE_F	_	30.5	_
_55	CTE_E	_	55	_

Tabla de símbolos

TEMAS ESPECIALES

- **Grupo 1 CANT**
(Chappe-larza-Palomino-Rodriguez-Schwab)

Calcula el valor de la función CANT y se lo asigna a un identificador.

Esta función toma como entrada una lista de expresiones y devuelve el valor que resulta de definir la cantidad de elementos de la lista que se múltiplos de 2. Los elementos de la lista están separados por comas (,) y la lista se encuentra entre []

La lista podría ser vacía en cuyo caso se emitirá un mensaje "La lista está vacía"

El resultado será asignado al ID del lado izquierdo de la asignación

Formato:

`id := CANT [lista]`

Ejemplos:

<code>resul := CANT [3])</code>	<code>/\$ asigna al id resul el valor 0 porque 3 no es múltiplo de 2 \$/</code>
<code>resul := CANT [1,2,3,4,5]</code>	<code>/\$ asigna al id resul el valor 2 porque 2 y 4 son múltiplos de 2 \$/</code>
<code>resul := CANT []</code>	<code>/\$ mensaje la lista esta vacía y se asigna 0 al id resul \$/</code>